

Security Analytics & Threat Detection

Poorva Shirolkar

Cybersecurity Analytics | SOC Investigation | Log Analysis | Machine Learning

1. Project Overview

This consolidated report combines the work completed across nine security analytics artifacts into one GitHub-ready project report. The project focuses on SOC-style investigation using log analysis, SIEM concepts, Python-based data analysis, Kibana, NetFlow review, machine learning classification, anomaly detection, and AI-assisted log analysis validation.

The purpose of this report is to present the project as a professional cybersecurity portfolio artifact without including raw datasets or restricted course files. The report emphasizes methodology, tools, analysis workflow, key findings, and security relevance for SOC Analyst, Cybersecurity Analyst, and Security Analytics roles.

2. Tools and Techniques

- Python for data cleaning, feature processing, analysis, and machine learning.
- pandas and NumPy for structured security-data analysis.
- scikit-learn for Logistic Regression, Random Forest, and MLP/neural-network classification.
- Kibana for NetFlow investigation, filtering, and dashboard-driven review.
- ipwhois enrichment to map external destinations and ASN/company context.
- SIEM concepts including log review, event correlation, anomaly detection, alert validation, and investigation reporting.
- AI-assisted log analysis using Gemini and Microsoft Co-Pilot followed by manual verification.

3. Data Sources Analyzed

The project analyzed multiple types of security-relevant data sources similar to what SOC analysts review during triage and threat investigation:

- Domain frequency and URL/domain records.
- Access logs containing employee IDs, actions, resource IDs, IP addresses, timestamps, and results.
- NetFlow records containing source/destination IPs, ports, protocols, packet counts, byte counts, flow states, and alert indicators.
- DNS, SSL/TLS, endpoint, Windows event, HTTP, fileinfo, and SMTP-style event data.
- Phishing and benign/Alexa-labeled website feature datasets for machine learning classification.

4. Domain Frequency Analysis

Objective

The domain frequency analysis was designed to identify domains that appear rarely in large URL datasets. Low-frequency domains can be useful for security investigation because phishing, malware, and command-and-control infrastructure often use domains that are not commonly observed in normal traffic.

Methodology

- Imported URL data using Python and pandas.
- Used domain extraction logic to isolate the primary domain from URLs.
- Calculated frequency values for each domain using a frequency counter.
- Filtered domains using a frequency threshold to identify uncommon or suspicious domains.

Key Results

- Processed a large URL dataset of approximately 1,000,000 records.
- Generated a domain-frequency table containing URL, domain name, and frequency score fields.
- Filtered low-frequency domains below the selected threshold, producing 156,871 records for further review.
- Demonstrated how frequency-based analysis can support threat hunting and suspicious-domain prioritization.

5. Access Log Analysis

Objective

The access log analysis focused on reviewing authentication and resource-access activity to identify suspicious patterns, unusual access behavior, and potentially risky access events.

Methodology

- Loaded access logs, employee details, and resource details into Python dataframes.
- Mapped employee names and departments to access-log records using employee IDs.
- Mapped resource departments to log entries using resource IDs.
- Enriched raw logs with user, department, resource, location, IP address, action, and result context.

Key Results

- Analyzed approximately 333,952 access-log records.
- Integrated supporting context from 100 employee records and 81 resource records.
- Created enriched log views to support user-behavior review, access-trend analysis, and authentication-event investigation.
- Demonstrated a workflow similar to SOC access review, where raw logs are enriched before triage and reporting.

6. NetFlow Analysis

Objective

The NetFlow analysis investigated outbound network activity to identify suspected machines that may require security review based on data-transfer behavior, destination classification, and connection patterns.

Methodology

- Used the ipwhois module to identify external IP ownership and ASN/company information.
- Updated the NetFlow JSON logs with source IP, destination IP, and ASN company fields.
- Imported the enriched log file into Kibana for filtering, visualization, and review.
- Focused on internal-to-external traffic where the destination ASN company was marked as NONE.

Kibana query used during investigation:

```
int_src_ip : true AND int_dest_ip : false AND ASNCompany : "NONE"
```

Key Results

- The Kibana query returned 148 external traffic results for review.
- The analysis focused on devices transmitting large amounts of data to external servers, particularly transfers above the investigation threshold.
- Suspicious activity included multiple large outbound transfers and unusually long-duration sessions.
- The investigation workflow demonstrated how NetFlow analysis can support potential data-exfiltration triage and prioritization.

7. AI-Assisted NetFlow Log Analysis

Objective

The AI log analysis project evaluated how large language models can assist with first-pass security analysis of NetFlow data. Gemini and Microsoft Co-Pilot were used to summarize traffic patterns, identify suspicious behavior, and suggest potential security risks. The AI outputs were then compared against manual analysis.

Dataset Summary

Metric	Observed value
Total records	165 entries
Fields	20 fields including timestamps, flow identifiers, IPs, ports, protocols, packet/byte counts, state, reason, and alert flags
Most active source IP	192.168.163.136 with 90 occurrences
Most active destination IP	72.21.91.29 with 40 occurrences
Common destination ports	80, 443, and 1900
Transport protocols	TCP dominated traffic, with UDP as a smaller portion
Application protocols observed	HTTP, TLS, DNS, fileinfo, and other event types
Alerted records	A small number of records were marked as alerted=True and required investigation

Key Suspicious Behaviors Identified

- Potential data exfiltration or command-and-control behavior involving unusual outbound traffic over port 1900.
- A flagged host with failed TLS/SSL activity and elevated outbound data transfer requiring endpoint investigation.
- Horizontal scanning behavior from 192.168.163.136 toward multiple destination IPs over common web ports.
- Protocol mismatches involving traffic on port 1900 that did not align with normal SSDP behavior.
- Potential lateral-movement indicators involving RPC/management ports that required contextual validation.

Gemini vs. Microsoft Co-Pilot Assessment

Evaluation area	Gemini	Microsoft Co-Pilot
Traffic summarization	Strong summary with useful suspicious-traffic indicators.	Good high-level summary of active IPs, ports, and protocols.
Threat detection precision	More accurate at highlighting credible threats such as horizontal scanning, abnormal port 1900 activity, and failed TLS sessions.	More likely to label routine activity as suspicious without sufficient filtering.
False positives	Produced fewer false positives after prompt tuning.	Generated more false positives, especially around routine update traffic and internal management flows.
Best use case	Useful first-pass threat investigation support when paired with manual validation.	Useful for baseline traffic summaries and broad trend review.

Final assessment: AI tools were useful for rapid pattern discovery and summarization, but all high-risk conclusions required manual validation using observable flow metrics, timing patterns, protocol behavior, and environmental context.

8. Machine Learning Classification - Logistic Regression

Objective

The first machine learning phase used Logistic Regression to classify malicious/phishing activity versus benign/Alexa traffic using structured web or page-behavior features.

Methodology

- Loaded the labeled dataset into Python using pandas.
- Mapped class labels into numeric values for supervised learning.
- Split the dataset into training and testing sets.
- Trained a Logistic Regression model and evaluated it with accuracy, confusion matrix, ROC curve, and ROC-AUC score.
- Reviewed model coefficients and feature importance to understand which variables influenced classification decisions.

Key Results

- The dataset contained approximately 24,232 records and 13 core features.
- The Logistic Regression model achieved approximately 82.9% accuracy.
- ROC-AUC was approximately 0.854, showing useful but moderate separation between benign and malicious classes.
- Important features included page interaction and script-related indicators such as inline counts, onload counts, onclick counts, and script-block measures.

9. Machine Learning Classification - Random Forest

Objective

The Random Forest model was tested to improve classification performance and better capture non-linear relationships in the dataset.

Methodology

- Trained Random Forest classifiers using different numbers of estimators.
- Compared model outputs using accuracy scores and confusion matrices.
- Reviewed feature-importance scores to understand the strongest classification signals.
- Compared Random Forest performance against Logistic Regression and neural-network models.

Key Results

- Random Forest models achieved approximately 91-92% accuracy across tests.
- Random Forest produced stronger classification performance than Logistic Regression in the project results.
- The model helped identify behavior-based features that contributed strongly to malicious/benign classification.
- Feature-importance outputs supported analyst interpretation by showing which variables had the greatest classification influence.

10. Machine Learning Classification - Neural Network / MLP

Objective

The neural-network phase used an MLP classifier to compare whether a neural-network approach could improve malicious-activity classification.

Methodology

- Loaded and preprocessed the labeled dataset.

- Standardized feature values before training.
- Trained MLP classifiers using different iteration values.
- Compared accuracy and confusion matrices across multiple runs.
- Observed convergence warnings and evaluated how optimizer limitations affected results.

Key Results

- The MLP/neural-network models achieved approximately 85-87% accuracy depending on training configuration.
- The neural-network model performed better than a basic baseline but did not outperform the Random Forest model.
- Convergence warnings indicated that additional tuning, scaling, or hyperparameter optimization would be needed for production-quality use.
- The experiment demonstrated the importance of validating ML models with multiple metrics rather than relying on accuracy alone.

11. Model Comparison and Lessons Learned

The machine learning experiments compared Logistic Regression, Random Forest, and neural-network/MLP classifiers for malicious versus benign activity classification.

- Logistic Regression achieved approximately 82.9% accuracy and a ROC-AUC of approximately 0.854. It provided an interpretable baseline through model coefficients and feature influence.
- Random Forest achieved approximately 91-92% accuracy and showed the strongest overall classification performance. It also provided feature-importance outputs that helped explain which behavior-based signals contributed most to classification.
- The MLP/neural-network classifier achieved approximately 85-87% accuracy depending on configuration. It demonstrated the value of model comparison, but convergence warnings showed that additional tuning would be required for stronger performance.
- Random Forest was the strongest model for the classification task based on the observed results.
- Logistic Regression remained useful because it provided interpretable coefficients and a simple baseline.
- Neural-network results showed that more complex models do not automatically outperform well-tuned traditional models.
- Feature analysis helped translate model outputs into security reasoning, which is important for analyst workflows.

12. SOC Investigation Workflow Demonstrated

Across all nine artifacts, the combined project demonstrates an end-to-end SOC-style workflow:

1. Collect and load security data from domains, access logs, NetFlow records, and labeled datasets.
2. Clean and enrich logs with contextual fields such as employee information, departments, resource departments, source/destination IPs, ports, and ASN/company information.
3. Identify suspicious events using filtering, baselining, frequency analysis, thresholds, SIEM-style correlation, and anomaly detection.
4. Investigate specific behaviors such as abnormal outbound traffic, failed TLS activity, suspicious port usage, horizontal scanning, and potential data exfiltration.
5. Validate AI-assisted findings against manual evidence to reduce false positives.
6. Train and evaluate machine learning models for malicious/benign classification.
7. Document findings in a format suitable for reporting, escalation, and analyst review.

13. Key Security Findings

- Low-frequency domain analysis can help prioritize domains for threat hunting and phishing investigation.
- Access-log enrichment improves analyst ability to connect activity to users, departments, resources, and access patterns.

- NetFlow analysis is valuable for detecting abnormal outbound traffic, protocol mismatches, unusual sessions, and potential data-exfiltration behavior.
- AI-assisted log analysis can accelerate first-pass review, but manual validation is required to confirm true threats and remove false positives.
- Machine learning can support malicious/benign classification, but model outputs must be interpreted carefully using confusion matrices, ROC curves, feature importance, and security context.
- SOC investigations benefit from combining multiple evidence sources rather than relying on a single alert or tool output.

14. Recommended Response Actions

- Investigate hosts involved in abnormal outbound traffic, especially when traffic includes high-volume transfers, long sessions, or protocol mismatches.
- Review failed TLS/SSL activity and alerted flow records for possible malware, command-and-control, or misconfigured applications.
- Restrict abnormal outbound traffic on ports that should normally remain local or internal, such as SSDP/port 1900.
- Validate management traffic such as RPC, WinRM, RDP, and SSH to ensure it is authorized and expected.
- Use baseline comparison to differentiate routine update/CDN traffic from suspicious beaconing or exfiltration behavior.
- Apply AI-assisted analysis only as a supporting tool and require manual evidence-based verification before escalation.